Supplement: Computational Complexity
Big-Oh and Little-Oh Notation

Brad Chattergoon
Industrial Engineering & Operations Research
University of California, Berkeley

August 6, 2026

This short supplement accompanies the *Mathematics Boot Camp* notes. It develops the language of computational complexity—in particular the *big-oh* (O) and *little-oh* (o) notation—which we use throughout the main notes, most notably to describe the remainder of a Taylor expansion. The reader who is comfortable with asymptotic notation can safely skip ahead.

1 Algorithmic Complexity

Algorithms are typically associated with computers and computer science, but they are actually a very general and non-specific concept.

Definition 1.1 (Algorithm)**.** An *algorithm* is a sequence of steps or instructions intended to solve some kind of problem.

The reader should be able to see that this is not specific to computing. As an example, a cooking recipe is an algorithm that solves the problem of how to create a particular food product. The recipe-as-algorithm framing also highlights something about algorithms more generally: they take in an input of some kind, and the algorithm acts as a sequence of steps that transforms the input into a target output. This framing is inherent to any discussion of algorithms, because an algorithm solves a problem that we must first specify.

Definition 1.2 (Problem)**.** A *problem* is a specified input–output pair.

An example of a problem is integer multiplication.

Problem:	Integer multiplication (positive)
Input:	Integers $x, y \in \mathbb{N}^+$
Output:	The product $x \cdot y$

Let us give two different algorithms to solve this problem. To make it concrete, let $x = 2$ and $y = 10$.

Algorithm 1. Use the view of multiplication as a sum of x replicates of y:

$$x \cdot y = 10 + 10 = 20.$$

Algorithm 2. The typical grade-school multiplication algorithm:

$$\begin{array}{r} 10 \\ \times\ 2 \\ \hline 00 \\ +\,20 \\ \hline 20 \end{array}$$

Both of these algorithms get to the same output, and we know this output is correct, but they have different *algorithmic complexity*.

Remark 1.3. Algorithmic correctness is an entire field of study in itself, but it is outside the scope of this note, where we simply try to establish an understanding of complexity. Additionally, algorithmic complexity is comprised of *space complexity*—how much intermediate data must be stored during the algorithm—as well as *run-time complexity*—how long it takes to process the inputs to get the outputs. Both use the same measurement idea, but we typically focus on run-time complexity.

1.1 A unit of computation

In order to measure anything we need a concept of a unit. We might be tempted to use something like real-world clock time, so the unit is seconds. We can quickly dissuade ourselves of this idea by noticing that clock time proxies algorithmic (time) complexity but also depends on the specific processor used. To compare different algorithms we would then need a single standardized processor for everyone to run their algorithm on—and we had better hope it lasts forever. That won't work.

Instead, let us think about what "processing speed" really is. A processor rated for x Hz is really saying it can perform x `flops`, or x "floating-point operations per second." The processor uses a `flop` as its base unit, and for algorithms we do something very similar: we take the base unit to be a *primitive operation*. On a real CPU a primitive operation is typically a floating-point operation (addition, subtraction, multiplication, division, among others), but we can be more abstract and define a primitive operation to be anything reasonable for the context. The only restriction is that each primitive operation should represent one unit of computation.

To illustrate algorithmic complexity in our example, let us define the following as primitive operations:

(i) Adding two single-digit numbers.
(ii) Multiplying two single-digit numbers.
(iii) Adding or removing a zero at the beginning or end of a number.
(iv) Determining which of two nonnegative integers is larger.

For concreteness, let m be the number of digits of x and n be the number of digits of y. Without loss of generality, let $m \leq n$.

2 Counting Primitive Operations

In this section we deliberately keep an explicit, near-exact count of the primitive operations each algorithm uses. This will be somewhat tedious—and that tedium is precisely the point. It will motivate the more economical notation we introduce afterward.

2.1 Adding two n-digit numbers

First let us establish how many operations it takes to add two n-digit numbers.

Problem: Add two n-digit nonnegative integers
Input: Integers $p, q \in \mathbb{N}^+$
Output: $p + q$

Write $p = p_n p_{n-1} \cdots p_2 p_1$ and similarly for q, and add digit by digit from the right, tracking a carry. The first digit gives

$$p_1 + q_1 = c_1 r_1, \qquad c_1 \in \{0, 1\},$$

which is one primitive operation, with c_1 a possible carry. For the second digit we compute $p_2 + q_2 + c_1 = c_2 r_2$: the sum $p_2 + q_2$ is one operation, adding the incoming carry c_1 is a second, and a third may be needed when $p_2 + q_2$ itself generates a carry (a sum of the form $1X$, whose combination with c_1 can carry again). This repeats for the remaining digits, so we use *at most* about $3n$ primitive operations.

Cost of addition: at most $3n$ primitive operations.

2.2 Algorithm 1: repeated addition

We now count the operations used by Algorithm 1, which computes $x \cdot y$ by repeatedly adding y:

(i) Find which of m or n is smaller, i.e. $\min(m, n)$ [1 op].
(ii) Since m is smaller, we replicate y and sum it x times.
(iii) Set $z = y$ [1 op], then

$$\text{for } i = 1, \ldots, x - 1: \quad z = z + y \qquad [\text{each addition} \le 3n \text{ ops}].$$

(iv) Return z.

The loop performs $x - 1$ additions, each costing at most $3n$ operations, plus the two setup operations, so the total is at most

$$1 + 1 + 3n(x - 1) = 3n(x - 1) + 2 \quad \text{primitive operations.}$$

Here is the crucial subtlety: $x - 1$ is the *value* of x, not its number of digits. An m-digit number can be as large as $10^m - 1$, so in the worst case $x - 1 = 10^m - 2$, and the count is at most

$$3n\,(10^m - 1) + 2.$$

As the number of digits m of x grows, this count grows in proportion to 10^m: each additional digit of x multiplies the number of additions by about ten.

Remark 2.1 (Value versus size). This is the classic distinction between the *value* of an input and its *size*. We measure complexity in the size of the input—the number of digits (equivalently, bits)—not its numeric value. The number of additions here, $x - 1$, is small relative to the value of x but grows like 10^m in the number of digits m; algorithms whose cost behaves this way are called *pseudo-polynomial*. Despite its simplicity, then, Algorithm 1 is not a practical multiplication algorithm.

2.3 Algorithm 2: grade-school multiplication

To count the operations in Algorithm 2 we first count the cost of multiplying an n-digit number by a single digit. We can reuse the idea of Algorithm 1: multiplying the n-digit number y by a single digit x_i is the same as adding y to itself x_i times. The difference is that here the multiplier is a single digit, whose value is at most 9, so we need at most 9 additions. Reusing the count above, this costs at most

$$1 + 9 \cdot 3n = 27n + 1 \quad \text{primitive operations.}$$

(The bound 9 is legitimate here precisely because a single digit has value at most $9 = 10^1 - 1$; contrast this with the m-digit multiplier of Algorithm 1, whose value can be as large as $10^m - 1$.)

Algorithm 2 then proceeds as follows:

(i) Determine which of m or n is smaller [1 op].

(ii) Choose the multiplicand to be the larger number $y = y_n \cdots y_1$ and the multiplier to be the smaller number $x = x_m \cdots x_1$.

(iii)–(iv) For each of the m digits x_i, form the partial product $f = y \times x_i$ (at most $27n + 1$ ops) and shift it left by up to $i - 1$ places using operation (iii) (at most m ops). This is at most $27n + m + 1$ ops per digit, hence at most $m(27n + m + 1)$ ops over all m digits.

(v)–(vii) Initialize $z = 0$ [1 op], then add the m partial products. Each such addition involves numbers with at most $n + m$ digits, costing at most $a\,n$ operations for some constant $a \geq 3$; summing the m partial products costs at most $a\,mn$ ops.

Collecting the counts, Algorithm 2 uses at most

$$1 + 1 + m(27n + m + 1) + a\,mn = (27 + a)mn + m^2 + m + 2 \quad \text{primitive operations.}$$

2.4 Comparing the two algorithms

Both algorithms produce the same correct product, but their operation counts behave very differently. To compare them we would like to express both in terms of a single variable. We are after an upper bound on the worst-case run time, but it need not be as tight as possible—only representative. Since one of m and n is at least as large as the other, we may set the smaller equal to the larger (when they are not already equal) and still have a valid, if slightly weaker, upper bound. Setting $m = n$ in this way keeps us honest about the worst case while sparing us some of the tedious bookkeeping, and it still yields an acceptably tight bound.

With $m = n$, Algorithm 2's count $(27+a)mn+m^2+m+2$ becomes $(27+a)n^2+n^2+n+2 = (28 + a)n^2 + n + 2$, and the two algorithms compare as

$$\text{Algorithm 1: at most } 3n\,(10^n - 1) + 2, \qquad \text{Algorithm 2: at most } (28 + a)n^2 + n + 2.$$

The first grows in proportion to 10^n as the number of digits grows, while the second is a polynomial in n. Even setting the exact constants aside, Algorithm 1's count explodes while Algorithm 2's grows like n^2, so Algorithm 2 is vastly more efficient.

Keeping exact counts like these, however, is both tedious and fragile. The precise constants—the 3, the 27, the a—are artifacts of exactly how we chose to define primitive operations; a different but equally reasonable choice would change them without changing which algorithm is better. What is robust, and what actually distinguishes the two algorithms, is the way each count *scales* as the input grows. That observation is the entire motivation for the notation we introduce next.

3 Big-Oh Notation

We now abstract away the constant factor and emphasize only the scaling. For the grade-school count $(28 + a)n^2 + n + 2$, the n^2 term dominates as n grows, and the exact leading constant is an artifact of our primitive-operation choices. In some textbooks the primitive operations are defined so that the same grade-school algorithm comes out to a bookkeeping result of $4n^2$ instead of the $(28 + a)n^2 + n + 2$ we obtained here—a different constant, but the same n^2 scaling. Rather than track these constants, we record only the scaling and write the count as "$\mathrm{O}(n^2)$."

Definition 3.1 (Big-Oh)**.** Let $h, g : \mathbb{N} \to \mathbb{R}^+$ be functions. We say $h = \mathrm{O}(g)$ ("big-oh of g") if there exist a constant $k > 0$ and an $N \in \mathbb{N}$ such that

$$h(n) \leq k \cdot g(n) \qquad \text{for all } n > N.$$

Example 3.2. Let $h(n) = 2n^2 + n - 1$ and $g(n) = n^2$. Then $h = \mathrm{O}(n^2)$: choosing $k = 3$, the inequality $2n^2 + n - 1 \leq 3n^2$ is equivalent to $n - 1 \leq n^2$, which holds for all $n \geq 1$. For instance, at $n = 2$,

$$h(2) = 2 \cdot 2^2 + 2 - 1 = 9 \leq 12 = 3 \cdot 2^2 = k\, g(2),$$

so $h(n) \leq 3\, g(n)$ for all $n \geq 1$.

Remark 3.3. The constant is not irrelevant in practice. An algorithm running in n^2 steps is about twice as fast as one running in $2n^2$ steps, even though both scale as n^2. Big-oh deliberately suppresses this constant to focus on scaling; it is a statement about behavior as the input grows large, not about a head-to-head race on a fixed input.

With this notation our two algorithms are summarized cleanly. The grade-school algorithm runs in $\mathrm{O}(n^2)$, since its count $(28 + a)n^2 + n + 2$ is bounded by a constant multiple of n^2 once n is large (Exercise 3.4). Algorithm 1's count $3n(10^n - 1) + 2$ is $\mathrm{O}(n \cdot 10^n)$—it grows in proportion to 10^n, exponentially in the number of digits—confirming that grade school is dramatically better.

Exercise 3.4. Find a constant $k > 0$ and a threshold $N \in \mathbb{N}$ such that $(28 + a)n^2 + n + 2 \leq k\, n^2$ for all $n > N$, thereby verifying directly that the grade-school count is $\mathrm{O}(n^2)$.

Once a problem is known to admit a polynomial-time algorithm, a natural next question is how small we can make the exponent. For integer multiplication, Karatsuba's algorithm improves the exponent to

$$\mathrm{O}(n^{\log_2 3}) = \mathrm{O}(n^{1.585}),$$

and the best known algorithms run in about $\mathrm{O}(n \log n)$. Each such improvement is an order-of-magnitude change in scaling, which for large inputs dwarfs any constant-factor difference.

4 Little-Oh Notation

The concept of big-oh is what we use for algorithmic complexity, and that is how we motivated it, but the idea of comparing functions by their scaling behavior is more general. Big-oh asks whether *some* constant k makes $h(n) \leq k\, g(n)$ hold for all large n. What if instead *every* constant $k > 0$ makes it hold for all sufficiently large n?

Definition 4.1 (Little-Oh)**.** Let $h, g : \mathbb{N} \to \mathbb{R}^+$ be functions. We say $h = \mathrm{o}(g)$ ("little-oh of g") if for *every* constant $k > 0$ there exists an $N_k \in \mathbb{N}$ such that

$$h(n) \leq k \cdot g(n) \qquad \text{for all } n > N_k.$$

Example 4.2. Let $h(n) = 10n^2$ and $g(n) = n^3$. For any $k > 0$, the inequality $10n^2 \leq kn^3$ is equivalent to $n \geq 10/k$, so it holds for all $n > 10/k$. Hence $h = \mathrm{o}(n^3)$. Note that $h = 10n^2$ is *not* $\mathrm{o}(n^2)$: there the ratio $h(n)/n^2 = 10$ is constant and does not shrink below every k.

This idea makes little-oh useful through the following equivalence.

Proposition 4.3. *Let $h, g : \mathbb{N} \to \mathbb{R}^+$ with $g(n) > 0$ for all large n. Then $h = \mathrm{o}(g)$ if and only if*

$$\lim_{n\to\infty} \frac{h(n)}{g(n)} = 0.$$

Proof. ($\Leftarrow$) Suppose $h(n)/g(n) \to 0$. Fix any $k > 0$. By the definition of the limit there is an N_k such that $h(n)/g(n) < k$ for all $n > N_k$; since $g(n) > 0$, this is $h(n) < k\, g(n)$, so $h = \mathrm{o}(g)$.

($\Rightarrow$) Suppose $h = \mathrm{o}(g)$. Fix any $\varepsilon > 0$ and apply the definition with $k = \varepsilon$: there is an N_ε such that $h(n) \leq \varepsilon\, g(n)$ for all $n > N_\varepsilon$, i.e. $0 \leq h(n)/g(n) \leq \varepsilon$. Since $\varepsilon > 0$ was arbitrary, $h(n)/g(n) \to 0$. □

Although we stated the definition for the limit $n \to \infty$ (the natural setting for complexity, where the input size grows), the same idea applies to any limiting process. This notation is especially useful when we want to denote a "tail" that goes to 0 faster than the terms we keep. This is exactly the role it plays in a Taylor expansion: writing

$$f(t + \Delta t) = f(t) + f'(t)\,\Delta t + \mathrm{o}(\Delta t) \qquad (\Delta t \to 0)$$

says that the error of the tangent-line approximation vanishes faster than the displacement Δt. Here the relevant limit is $\Delta t \to 0$ rather than $n \to \infty$, and $h = \mathrm{o}(g)$ means $h(\Delta t)/g(\Delta t) \to 0$ as $\Delta t \to 0$; the interpretation is otherwise identical.